Chapter 2 — Project Setup

Chapter 2 — Project Setup

> **From an empty folder to a running dev server.** No React yet — but by the end you'll have a Next.js + TypeScript + Tailwind app booting at `http://localhost:3000`, with your `lib/` folder full of typed data and pure scoring logic.

What you'll learn

- **Why Next.js** is the right framework for this app (and when it would be the wrong one).
- **What every config file does** — `package.json`, `tsconfig.json`, `tailwind.config.ts`, `next.config.mjs`, `.eslintrc.json`. No more "I just copy-paste these from someone."
- The Next.js **App Router** mental model: file-based routing, `app/` directory layout, what `layout.tsx` is, what `page.tsx` is.
- How to **transcribe data into TypeScript files** without losing your mind to bracket-matching errors.
- The npm install / `npm run dev` workflow, and what to do when something breaks.

Lessons in this chapter

1. **01-why-nextjs-typescript.md** — The framework decision tree. Next.js vs Vite vs Create-React-App vs plain HTML. Why we picked what we picked, and what would change if the requirements were different.
2. **02-scaffolding-the-project.md** — Run-by-run walkthrough of creating the project from scratch. Every file. Every line. Every config option, explained.

3. **03-creating-data-files.md** — Filling out `lib/matches.ts`, `lib/teams.ts`, `lib/players.ts` with real data. The boring lesson where you'll spend the most keystrokes.

Files you'll create in this chapter

```
package.json                ← dependencies
package-lock.json           ← (generated by npm)
tsconfig.json               ← TypeScript compiler config
next.config.mjs             ← Next.js config
tailwind.config.ts          ← Tailwind theme config
postcss.config.mjs          ← PostCSS pipeline (used by Tailwind)
.eslintrc.json              ← ESLint rules
.prettierrc                 ← Prettier formatting rules
.gitignore                  ← what git ignores
README.md                   ← top-level project README
app/
  layout.tsx                ← root HTML wrapper (server component)
  page.tsx                  ← the home route
  globals.css               ← Tailwind directives + global styles
lib/
  types.ts                  ← (Chapter 1 Lesson 2)
  scoring.ts                ← (Chapter 1 Lesson 3)
  matches.ts                ← all 31 match objects
  teams.ts                  ← all 8 team objects with picks
  players.ts                ← PLAYER_INFO dictionary
  constants.ts              ← shared style helpers + ball colors
```

New React/Next.js concepts introduced

- **App Router** — Next.js 13+ file-based routing system.
- **Server components** (the default) — components that render on the server, no JavaScript shipped to the browser.
- **`'use client'` directive** — opt a single file into client-side rendering. Foreshadowed here, used in earnest in Chapter 4.
- **TypeScript path aliases** (`@/components/...`) — clean imports without

`../../../`.

- **Tailwind CSS** — utility-first styling. We won't use it heavily (the codebase uses inline styles), but we set it up.

How long it should take

- 30 min reading per lesson
- 1.5–2 hours typing the data into Lesson 3 (the data is real, the file is long)
- ~3.5 hours total

Before you start

You need:

- Node.js 20+ installed (`node --version`)
- A terminal (Windows users: PowerShell is fine, the lessons assume PowerShell commands)
- ~30 minutes for `npm install` to finish on a fresh project (it's about 400 packages)

If you're following along with this repo as the reference, you can `cd` into a *new* empty folder and rebuild the project there, then `diff` against this repo's state. Or just read along and sanity-check against the live files in `package.json`, `app/`, `lib/`.

Open **01-why-nextjs-typescript.md** when ready.